

Dossier créatif ENJMIN 2020

Parcours programmation

Par Simon Moisan



Sommaire :

Partie commune

1.	Analyse de l'oeuvre	Page 1
2.	Principe du jeu	Page 2
3.	Navigation	Page 3
4.	Partis pris	Page 4
5.	Originalité	Page 4
6.	Inspiration	Page 5
7.	Bibliographie	Page 5

Partie spécifique

I.	Portfolio	Page 6
1.	Projets universitaires	Page 6
2.	Projets personnels	Page 7
II.	Problème technique	Page 10
1.	Motivations	Page 10
2.	Les performances	Page 10
3.	Cas n°1 : Gérer un grand nombre d'objet	Page 11
4.	Cas n°2 : Gérer une scène vaste et complexe	Page 12
5.	Conclusion	Page 15

Ce document contient le dossier créatif pour ma candidature au Master JMIN pour l'année 2020. La partie commune comportera une analyse du film "La Jetée" de Chris Marker. Elle sera suivie d'une proposition de jeu vidéo inspirés des éléments de réflexion acquis pendant l'analyse.

Analyse de l'oeuvre :

"La Jetée", est un film français de 28 minutes qui a été créé par Chris Marker en 1962. Ce film est un diaporama composé d'une succession de photographies en noir et blanc, d'un unique narrateur racontant l'intrigue et d'une bande son. Le film est considéré comme un chef d'œuvre par de nombreuses critiques et réalisateurs. Cela fait de lui une source d'inspiration pour des œuvres plus récentes comme "L'armée des douzes singes", qui reprend une intrigue similaire. La particularité de ce film réside aussi bien dans les choix techniques, que derrière sa mise en scène et que l'intrigue en elle-même.

Le film nous fait suivre une histoire à travers les yeux et les souvenirs d'un homme, le protagoniste. L'intrigue gravite autour d'un de ses souvenirs; lui, quand il était enfant sur la jetée d'Orly, juste avant le début de la troisième guerre mondiale. Ce souvenir d'une grande netteté, contient deux éléments importants : le visage d'une femme et la mort d'un homme. C'est grâce à ce souvenir, que le protagoniste est choisi par des scientifiques parmi les survivants de la guerre, pour participer à une expérience qui consiste à voyager à travers le temps. Le but de cette expérience est de pouvoir "Appeler au secours du présent, le passé et le futur", dans le but de faire transiter des ressources pour la survie de l'humanité. Le souvenir de la jetée, et plus particulièrement du visage de la femme, lui sert de point d'ancrage à travers le temps pour ne pas succomber à la folie.

Ainsi, le film nous introduit à la thématique du voyage temporel en utilisant des souvenirs comme base. C'est à travers cette logique que le choix d'utiliser des photographies, à la place d'images en mouvement, prend tout son sens. En effet, les photographies permettent de symboliser des souvenirs du passé tel que nous nous les remémorons, c'est-à-dire fixes mais aussi incomplets. Ainsi, les événements de l'intrigue et le cours du temps lui-même est conçu de manière figée et irréversible. Cette idée peut être démontrée par le fait que les scientifiques cherchent à envoyer une personne dans le passé, non pas pour empêcher la guerre, mais pour faire survivre le présent.

Enfin, cette perception du temps met en avant un élément présent de plusieurs aspects du film : la boucle. On retrouve une première boucle par rapport à l'intrigue, avec la mort de l'homme qui a marqué le personnage principal. Cette même mort qui se trouve être la sienne lorsqu'il revient dans le passé sur la jetée d'Orly à la fin du film. C'est le souvenir de ce même événement qui l'a incité à faire ce dernier voyage dans le passé à ce moment-là. Une autre boucle, encore liée à l'intrigue, a lieu lorsque le personnage principal va dans le futur, et non plus dans le passé. En effet, dans le futur l'humanité a survécu et offre au protagoniste un réacteur électrique pour sauver son monde. Ainsi, pour que le futur survive, il est obligé d'aider son propre passé, qui correspond au présent.

Postulat : Si les voyages temporels ne peuvent changer le cours du temps, alors ce dernier est immuable.

Le principe du jeu

Il existe une infinité d'univers possibles, l'écoulement du temps de chacun de ces univers porte le nom de "ligne temporelle". Toutes les lignes se ressemblent, mais sont toutes différentes et sont par nature stables et prédéfinies. Chacune d'entre elles est séparée en plusieurs segments par des nœuds temporels. Cependant, il arrive que des voyageurs temporels voyagent dans des lignes temporelles parallèles à la leur. En plus de déstabiliser des segments temporels, leurs actions peuvent avoir des conséquences plus graves. En effet, elles peuvent empêcher la réalisation d'événements clés de la ligne temporelle, ce qui peut la fragiliser, voire même la détruire.

Pour éviter cela, le Maître du Temps a créé les Time Fixer, des entités hors du temps dont l'objectif est de "réparer" ces lignes temporelles endommagées. Dans le jeu, le joueur incarne un Time Fixer, et va interagir avec différentes lignes temporelles réparties dans des chapitres. Dans chaque chapitre, le joueur va interagir entre le moment où le voyageur temporel agit et l'événement clé qui a été annulé. Pendant cet intervalle, le joueur doit réaliser une série d'actions pour faire en sorte que l'événement clé ait bel et bien lieu. Ces "actions" faites par le joueur auront un impact direct sur la ligne temporelle. Une action peut être aussi anodine que de verrouiller une porte. Mais elle peut être bien plus importante, comme le fait d'assassiner un personnage historique. Le joueur pourra choisir entre plusieurs séries d'actions différentes, aussi bien en termes de difficulté que de choix moraux. Pour l'aider, il aura accès à un arsenal de pouvoirs et de capacités lui permettant de progresser dans son environnement et d'identifier les actions qui vont réparer la ligne temporelle.

Légende :

	Segment stable		Modification d'un voyageur temporel
	Segment instable		Événement clé annulé
	Segment modifié par le Time Fixer		Actions faites par le Time Fixer
	Nœud temporel		Événement clé rétabli

Pour que le principe soit suffisamment clair, je vais prendre un exemple concret. Prenons le cas d'une ligne temporelle parallèle et très similaire à la nôtre, à l'époque où le président Abraham Lincoln est encore en vie, entre le 1er et le 15 Avril 1865 (date de son assassinat). Cependant, un voyageur temporel venant d'une autre ligne temporelle, voyage dans le temps et tue John Wilkes Booth, l'assassin du président, fin Mars 1865. Avec la mort de John, celle d'Abraham Lincoln n'arrivera jamais dans cette ligne temporelle. Comme ce bouleversement risque de détruire la ligne temporelle, un Time Fixer doit intervenir entre la mort de John et celle supposée du président, c'est-à-dire entre le 1er Avril et le 15 Avril 1865. Pendant cet intervalle de temps, le Time Fixer devra trouver un moyen pour que la mort d'Abraham Lincoln ait bel et bien lieu le 15 Avril 1865 au théâtre Ford pendant la représentation de la pièce *Our American Cousin*. Pour cela, le Time Fixer a le choix entre plusieurs approches pour que l'événement clé se réalise, cependant chaque approche n'aura pas la même durée ou aura plus ou moins d'impact sur la ligne temporelle.

Ligne temporelle avant intervention du Time Fixer :



Ligne temporelle après intervention du Time Fixer :



La navigation

Le joueur contrôle le Time Fixer en vue à la troisième personne, avec un clavier et une souris, ou bien avec une manette. J'ai opté pour un gameplay asymétrique basé sur deux formes que le joueur peut incarner selon la situation :

La forme physique : Elle correspond au mode de déplacement par défaut du Time Fixer, ce qui lui permet de courir, sauter, escalader et aussi se camoufler dans l'obscurité. C'est sous cette forme que le joueur pourra faire des actions directes qui auront de l'impact sur la ligne temporelle en la modifiant. Un aspect important de cette forme est la discrétion et la vulnérabilité aux attaques. En effet, le joueur doit éviter à tout prix de se faire voir par quiconque. Si jamais le joueur se fait repérer, il peut rattraper le témoin pour l'étourdir et lui effacer la mémoire, ce qui est assez lent, mais plus sûr. Une autre option plus rapide mais qui peut limiter les actions du joueur plus tard, est de tuer le témoin et de faire disparaître le corps.

La forme spectrale : Pour atteindre cette forme, le Time Fixer sort de son propre corps sous la forme d'un spectre. Cela permet de profiter d'une nouvelle manière d'appréhender le jeu. Ainsi, le joueur devient invisible et peut léviter au prix d'un champ d'action limité autour de sa forme physique, qui reste immobile. Cette forme est très importante pour l'exploration et pour faire du repérage, en marquant divers points d'intérêts. L'accès à certains passages est possible sous cette forme, comme des conduits d'aérations ou des grilles. C'est aussi sous cette forme que le joueur aura accès aux capacités liées au temps en se plaçant à certains endroits spécifiques, les "noeuds temporels".

Les interactions avec le temps : Pendant un chapitre, le joueur pourra évoluer dans une ligne temporelle, dans une zone et un intervalle de temps prédéfini plus ou moins grand. Le début correspond à un moment après la modification du voyageur temporel, là où la ligne temporelle est encore stable, et la fin correspond à l'événement clé qui doit avoir lieu pour éviter la rupture de la ligne temporelle. Le joueur pourra interagir avec le temps dans cet intervalle et s'y déplacer en utilisant des nœuds temporels qui peuvent être trouvés à divers endroits de la carte. En se plaçant sur ces nœuds avec la forme physique, le joueur peut voyager d'un nœud à l'autre aussi bien dans l'espace (d'un endroit de la carte à un autre) que dans le temps (d'un moment à un autre). En se plaçant sur un de ces nœuds avec la forme spectrale, le joueur peut "naviguer" dans le segment temporel entre deux nœuds, en s'arrêtant, en l'accélérant ou en le rembobinant. Cela permet de récupérer des informations et des indices sur des pistes à exploiter pour remplir les différents objectifs qui permettront de réparer la ligne temporelle.

Un autre aspect important du gameplay, sont les actions réalisées par le joueur dans la ligne temporelle. En effet, chaque action faite avec la forme physique dans un segment va influencer la ligne temporelle et lui faire prendre une direction. Mais si jamais le joueur revient dans un nœud antérieur avec la forme physique, toutes les actions faites par le joueur dans le segment seront annulées pour éviter les paradoxes temporels. Même si cela force le joueur à réaliser ses actions sur la ligne temporelle dans l'ordre chronologique, cela doit aussi l'inciter à utiliser la forme spectrale pour explorer les différents segments et identifier au maximum les chemins possibles. En plus, cela lui permet aussi de modifier ou de refaire ses actions si jamais il veut changer d'approche.

La parti pris

L'idée de Time Fixer m'est venue à partir d'un effet dramatique particulier qui était mis en avant dans le film La Jetée : **le temps finit toujours par nous rattraper**. C'est cette fatalité qui s'abat sur le protagoniste lorsqu'il se fait tuer au moment où il est sur le point de rejoindre la femme qu'il a vu étant enfant sur la jetée d'Orly. Ainsi, même en utilisant le voyage dans le temps pour la revoir et échapper à la mort, son destin finit par le rattraper. C'est cette même mort qui va créer le traumatisme chez lui étant enfant et répéter la boucle narrative.

Cela m'a donné l'idée de personnifier cette fatalité sous la forme d'une entité à la fois détachée du temps, mais aussi dépourvue d'humanité : le Time Fixer. Son rôle est de s'assurer que le temps soit perturber le moins possible et qu'aucune ligne temporelle ne soit détruite à cause de ces perturbations. Par ailleurs, même si le Time Fixer contribue lui aussi à perturber l'espace temps en voulant le réparer. Il est soumis à des règles visant à limiter l'impact de ses actions, pour éviter qu'il détruise lui-même la ligne temporelle. De plus, il existe une infinité de lignes temporelles, ce qui implique que les actions faites par le Time Fixer vont forcément avoir lieu dans au moins l'une d'entre elles.

Si on veut comparer ce jeu à l'œuvre, on peut voir la mort du personnage principal comme l'événement clé, et son tueur comme étant le Time Fixer.

Originalité

Le thème du voyage dans le temps est souvent utilisé, aussi bien dans les films que dans les jeux vidéo. En effet, on retrouve les voyages temporels comme élément narratif, mais aussi comme élément de gameplay. En plus d'être un thème assez récurrent dans les jeux vidéos, il peut être difficile à intégrer car il peut être plein de failles s'il n'est pas suffisamment réfléchi. Si c'est le cas, cela risque de briser la suspension consentie de la crédulité du joueur et grandement nuire à l'expérience de jeu et à l'immersion. Il y a trois points qui font de Time Fixer un jeu original, malgré l'utilisation de la thématique des voyages dans le temps.

Le premier point est la manière dont est abordé le voyage temporel. En effet, le jeu veut mettre en avant les conséquences qu'impliquent le voyage temporel et qu'il ne doit être utilisé que pour empêcher l'effondrement d'une ligne temporelle.

Ensuite, le deuxième point concerne le rôle qu'incarne le joueur. Puisque ce dernier a pour rôle de "lutter" contre les voyageurs temporels et les perturbations qu'ils provoquent.

Enfin, l'environnement dans lequel le joueur évolue est assez vaste et complexe pour offrir au joueur suffisamment de liberté, ainsi que des opportunités et des approches différentes pour terminer un chapitre. Pour amplifier cela, le jeu met l'accent sur une 4ème dimension pouvant être explorée par le joueur : le temps. Cette dimension peut être utilisée pour se déplacer dans l'espace, d'un point à l'autre de la carte; et dans le temps, entre deux nœuds temporels.

Les inspirations

Pour les déplacements de la forme physique et l'aspect discrétion :



Dishonored 1



Tom clancy's splinter cell blacklist

Pour les interactions avec le temps, l'aspect exploration et les enquêtes :



Return of the Obra Dinn



Invisible Hours

Pour le changement entre la forme physique et spectrale du Time Fixer :



Luigi's mansion 3

Bibliographie

Analyse du film :

<https://www.cineclubdecaen.com/realisat/marker/jetee.htm>

https://fr.wikipedia.org/wiki/L%27Arm%C3%A9e_des_douze_singes

<https://www.arte.tv/fr/videos/060738-132-A/blow-up-quand-la-jetee-croise-vertigo/>

Recherches sur des oeuvre vidéoludiques :

https://en.wikipedia.org/wiki/List_of_games_containing_time_travel

https://www.youtube.com/watch?v=Gw_R_xDOUfQ

Jeux qui ont servi d'inspiration :

<https://dishonored.bethesda.net/fr>

<https://www.ubisoft.com/fr-fr/game/splinter-cell-blacklist/>

<https://obradinn.com/>

<http://www.theinvisiblehours.com/en>

<https://www.nintendo.fr/Jeux/Nintendo-Switch/Luigi-s-Mansion-3-1437312.html>

Portfolio

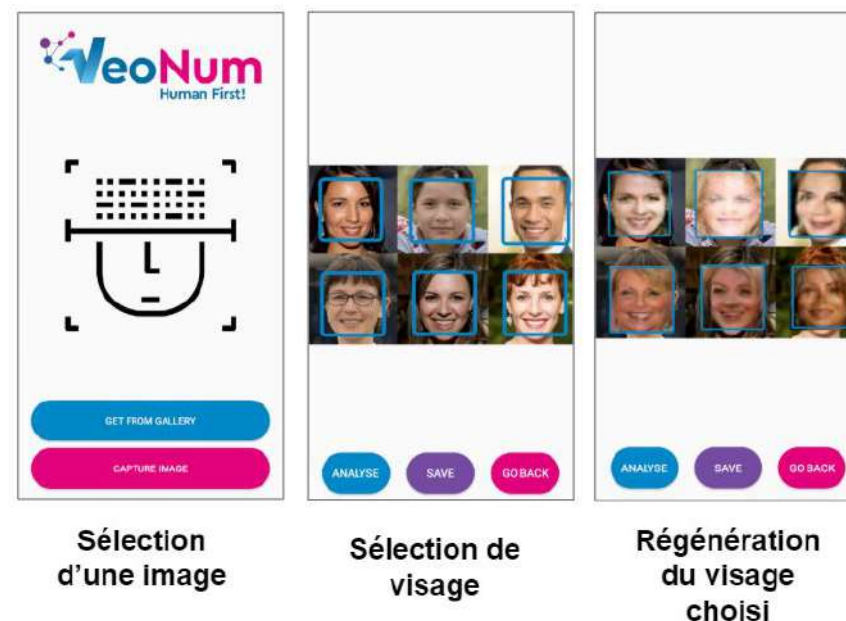
Projets universitaires

Projet tutoré : Projet tutoré de DUT qui consistait à intégrer plusieurs capteurs à une Raspberry PI, pour les faire interagir avec le jeu vidéo Minecraft, à travers un programme en Python. Les capteurs utilisés sont :

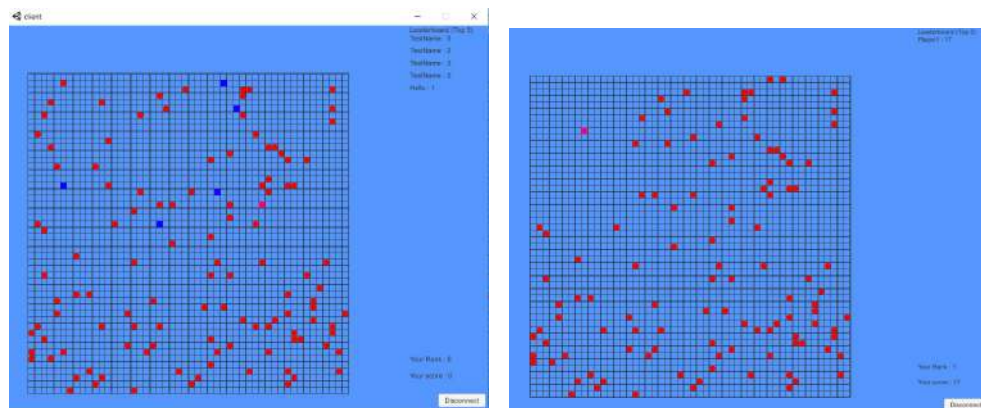
- Deux boutons poussoirs, le premier permettant de faire apparaître automatiquement une maison préprogrammée à l'emplacement du joueur. Le deuxième bouton fait apparaître un château avec un pont levé fait de blocs juste devant le joueur.
- Un capteur de température qui crée un afficheur fait de blocs qui indique la température détectée par le capteur en temps réel.
- Un capteur magnétique permet "d'animer" le pont de levés du château pour qu'il s'ouvre ou se ferme lorsqu'on fait passer une carte magnétique devant le capteur.

Simulation de fourmilière : Programme en C++ fait en première année de cycle ingénieur à l'ESIR. Le programme simule le comportement d'une fourmilière et de ses fourmis qui vont explorer une carte contenant des tas de nourriture. Les fourmis se déplacent de manière aléatoire sur la carte en laissant un peu de phéromones. Quand elles trouvent un tas de nourriture, elles vont s'y diriger pour y récupérer de la nourriture et la ramener à la fourmilière. Quand elles transportent de la nourriture, elles laissent derrière elles de plus grandes quantités de phéromones. Ainsi, on se retrouve avec des "chemins" de phéromones qui vont guider les autres fourmis aux tas de nourriture à proximité de la fourmilière.

Projet industriel : Projet réalisé pendant la 3ème année à l'ESIR. Il a consisté à créer une application Android permettant de régénérer des visages dans des images, à partir d'un Réseau Antagoniste Génératif (GAN). L'utilisateur peut soit prendre une photo avec son téléphone, soit en importer une depuis sa galerie pour y faire détecter tous les visages qui s'y trouvent. L'utilisateur peut alors sélectionner n'importe quel visage pour en générer un nouveau, qui est créé à partir du GAN. Le visage généré, qui n'appartient à personne, va remplacer celui d'origine sur l'image. L'utilisateur peut ensuite sauvegarder l'image modifiée dans son téléphone, ou répéter la régénération jusqu'à ce que le visage généré lui plaise.

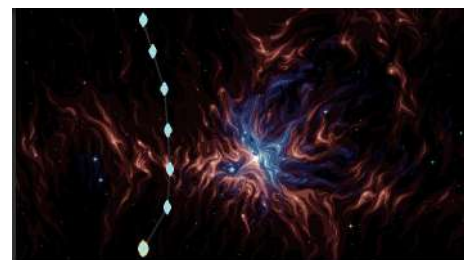


Projet de jeu en système réparti : Projet monitoré réalisé en 3ème année à l'ESIR, qui a consisté à créer un mini-jeu multijoueur. Le but de chaque joueur est de déplacer son personnage sur une grille contenant des bonbons. La partie se termine lorsque tous les bonbons ont été ramassés, le gagnant est le joueur ayant le plus de bonbons. La partie serveur du projet, codée en langage Go, s'occupe de gérer l'état global de la partie en envoyant à tous les clients l'état actuel de la partie. Cet état est mis à jour dès qu'un joueur se déplace sur une case. La partie client quant à elle gère l'affichage de la carte, du pion du client (carré rose), des bonbons (carrés rouge), des pions des autres clients (carrés bleu) et des scores, mais aussi les contrôles.



Projets personnels

Jeu vidéo Unity 2D "Space Battleship Shooter" : Un jeu de type "space shooter" où le joueur, au lieu de contrôler un petit vaisseau agile et rapide, va être aux commandes d'un puissant croiseur de combat équipé de tourelles, d'un laser géant et d'un bouclier énergétique. Il va devoir traverser plusieurs secteurs dans lesquels vont déferler des vagues d'ennemis à détruire. A la fin de chaque secteur, les tourelles équipées pourront être modifiées pour que le joueur puisse adapter son style de combat. Chaque tourelle possède ses propres spécificités (offensive, défensive, efficace contre des petits ennemis ou des plus gros) et caractéristiques qui peuvent être améliorées grâce à des points récupérés en détruisant les ennemis.

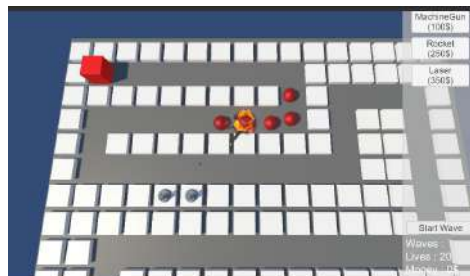
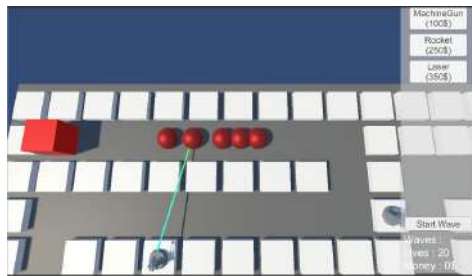


Jeu vidéo Unity 3D “FPS sandbox” : Environnement de test Unity utilisé pour développer différentes fonctionnalités propres à un jeu de tir à la première personne. Les fonctionnalités présentes sont :

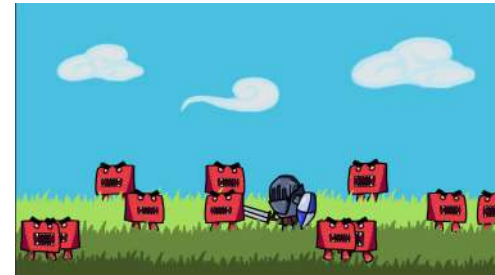
- Le déplacement du joueur et la gestion des collisions.
- Le fait de tirer avec des armes à feu (impacts des balles, application de force et de dégâts sur des caisses).
- Changement d’arme et rechargement des munitions de l’arme.
- Lancer de grenade (explosion et application de forces aux objets touchés).



Jeu vidéo Unity 3D “Tower Defence” : Jeu de tower defence standard avec des ennemis qui doivent atteindre la fin du niveau. Pour les empêcher, le joueur peut acheter et placer trois types de tourelle : mitrailleuse, lance rocket et laser.

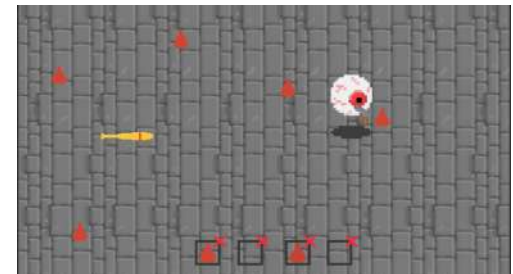
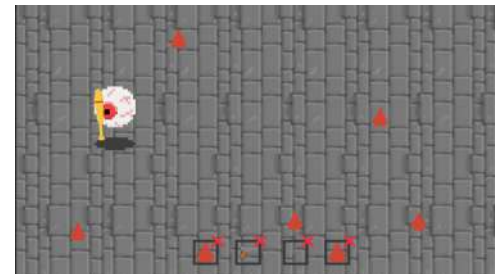


Jeu vidéo Unity 2D “Smash knight” : Petit jeu de combat en 2D où il faut affronter des vagues d’ennemis sur 3 plans différents sans se faire toucher.



Jeu vidéo Unity 2D “Top Down Shooter sandbox” : Environnement de test Unity utilisé pour développer des éléments de gameplay propre à un jeu en vue de dessus en 2D. Les fonctionnalités présentes sont :

- Le déplacement du joueur et l'animation du sprite.
- Un système d’inventaire : le joueur peut ramasser des potions et deux armes dans 4 emplacements d’inventaire.
- Le joueur peut utiliser une potion ou équiper une arme en cliquant sur l’icône dans l’inventaire.
- Le joueur peut déposer un objet de son inventaire en cliquant sur la croix rouge au-dessus de l’icône de l’objet.



Jeu vidéo Unity 3D “Top Down Shooter sandbox” : Environnement similaire à celui vu précédemment, mais en 3D. Les fonctionnalités présentes sont :

- Le déplacement et les collisions du personnage.
- La possibilité de tirer, soit en étant immobile avec une cadence de tir rapide, soit en marchant, mais la vitesse de déplacement et la cadence de tir sont plus lents.
- Des ennemis qui poursuivent le joueur (utilisation du pathfinding) et l'attaque quand ils sont à portés.
- Une caméra réglée de sorte à avoir en visuel le joueur et les ennemis présents sur la carte.

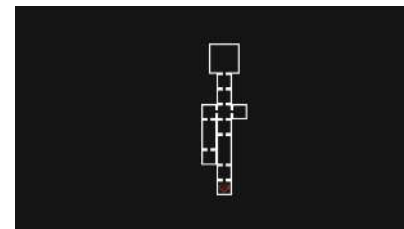
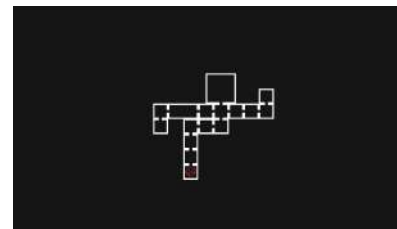


Générateurs aléatoires sur unity :

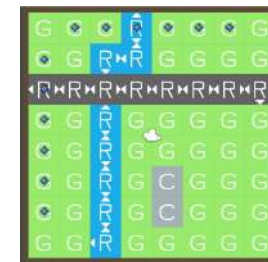
Un générateur de caverne en 3D en vue du dessus : Il génère un ensemble de grottes de tailles aléatoires qui sont reliées entre elles par des galeries. La caverne peut être générée plusieurs fois. La taille et la “densité” des galeries et des grottes peut être modifiée.



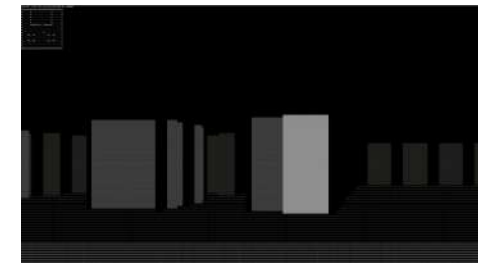
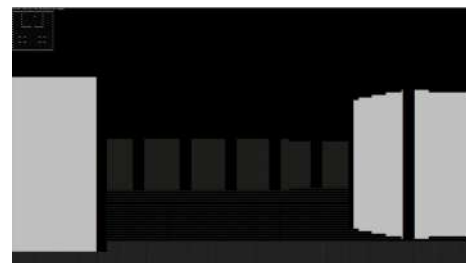
Un générateur de donjon de type Rogue-like en 2D : Les cases représentent les salles d'un donjon en vue du dessus. La génération peut être configurée en modifiant les salles qui peuvent apparaître, ainsi que leur probabilité d'apparition.



Un générateur de carte en 2D : la carte est générée sous la forme d'une grille où chaque case correspond à un environnement différent (G = plaines, R (bleu) = rivière, R (gris) = route, C = ville). La rivière et la route sont générées de manière à traverser la carte. Les cases de ville de sorte à ce qu'elles soient toujours collées les unes aux autres.



Jeu expérimental FPS ASCII : jeu à la première personne fonctionnant avec le terminal de commande windows. Dans ce jeu, le joueur explore un dédale de couloirs et de pièces, dont le rendu est fait en caractères ASCII. Les caractères sont placés à l'écran de manière à faire apparaître des murs et le sol. Le décors change en fonction de la distance entre lui et le joueur pour donner un effet de profondeur et simuler de la 3D.



Problème technique

Le problème technique que j'ai décidé d'aborder, est le suivant : **Comment optimiser les performances d'un jeu vidéo sur Unity ?**

Je suis bien conscient que cette problématique est très large et aussi qu'il est impossible de l'aborder dans son entièreté en seulement quelques pages. C'est pour cela que je vais en parler à travers deux cas concrets d'optimisation des performances et ce uniquement sur le logiciel Unity. Si j'ai choisi de concentrer ma problématique sur un logiciel en particulier, c'est parce que j'ai déjà acquis pas mal d'expérience et de connaissances sur ce dernier. En plus, cela va me permettre de mettre en pratique les techniques que je vais aborder dans des scènes Unity.

Les motivations

Il existe plusieurs motivations derrière l'optimisation des performances dans les jeux vidéo. Elle permet de fournir une meilleure expérience de jeu au joueur, en lui offrant par exemple une meilleure résolution, une meilleure fluidité ou une meilleure stabilité en jeu. Cela permet aussi de réduire les ressources nécessaires pour faire fonctionner un jeu, permettant ainsi une plus grande accessibilité. Un éditeur de jeux vidéo peut vouloir améliorer les performances de ces jeux pour plusieurs raisons, comme par exemple pour :

- Avoir de meilleures mécaniques de jeu ou de nouvelles mécaniques qui nécessitent beaucoup de ressources pour être fonctionnelles.
- Avoir de meilleurs effets graphiques sur l'eau, la lumière, des particules, des textures réalistes ou des shaders complexes.
- Gérer de nombreux objets et leurs comportements en jeu.
- Offrir un monde ouvert, vaste et complexe au joueur.
- Permettre à des consoles ou des configurations plus modestes de faire fonctionner le jeu, tout en ayant une qualité visuelle acceptable.

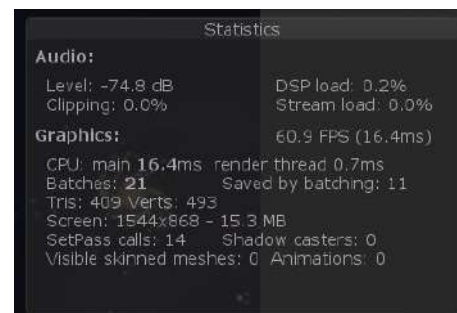
Les performances

Pour qu'un jeu vidéo fonctionne, il utilise des ressources provenant de deux composants électroniques : le CPU et le GPU.

Le CPU (Central Processing Unit), s'occupe de gérer les commandes envoyées par le joueur, les collisions, la physique et les divers scripts d'un jeu. De cette manière le CPU détermine qu'est qui doit être affiché et comment. Ces informations sont ensuite envoyées au GPU (Graphical Processing Unit), qui lui gère l'affichage de l'image à l'écran.

Des problèmes de performances peuvent provenir au niveau d'un de ces deux composants. C'est pourquoi pour résoudre ces problèmes, il est important d'identifier d'où ils viennent et y appliquer une solution adéquate.

De manière générale, quand on parle de performance dans un jeu vidéo, la plupart des gens pensent à des indicateurs comme la quantité de IPS (Images Par Seconde), le ping pour les jeux en ligne, ou plus vaguement la qualité graphique. Même si ces indicateurs sont importants à prendre en compte, ils ne sont pas très explicites et représentatifs des performances d'un jeu. Sur Unity, il est possible de voir ces indicateurs grâce à l'onglet "stats" se trouvant sur la fenêtre de jeu.

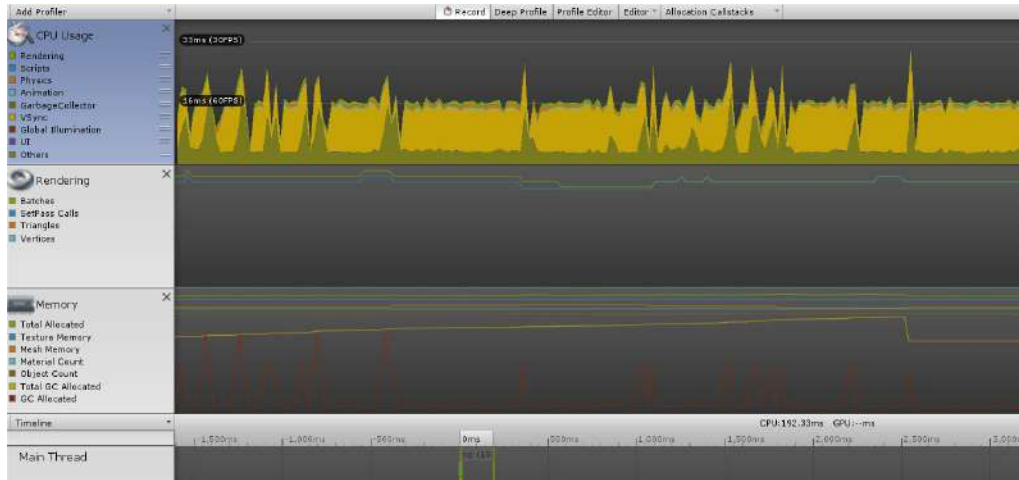


Parmis ces informations, les plus importantes sont :

Les "batches", qui sont des blocs de mémoire où sont stockés plusieurs GameObjects.

Les "tris" et "verts" qui correspondent aux nombres de triangles et de sommets modélisés dans le jeu.

Cependant, pour avoir un meilleur aperçu et mesurer ces performances plus en détail, il existe un autre outil sur Unity. Cet outil est le profiler, il permet d'avoir un compte rendu plus détaillé des performances du jeu en temps réel sous la forme de graphiques répartis dans plusieurs catégories.



Il permet de visualiser concrètement la quantité de ressources prises par chaque composant du jeu. De plus, il est très utile lorsque l'on veut appliquer des techniques d'optimisation, car il permet de voir si elles sont efficaces ou non, et à quel point. Il existe de nombreux indicateurs, certains sont plus utiles que d'autres selon le jeu, mais aussi selon ce que l'on cherche à optimiser. Les plus importantes étant :

- L'usage du CPU : correspond à la puissance de calcul allouée au jeu, il est subdivisé en plusieurs sous-catégorie : le rendering (pour le rendu graphique), le script (pour le code), la physique, l'animation, etc...
- L'usage mémoire (VRAM usage) : il augmente avec le nombre d'objets créés dans le jeu, mais aussi avec la qualité des textures, des matériaux et des shaders.
- Le rendering : qui indique entre autres, le nombre de triangles et de sommets modélisés.

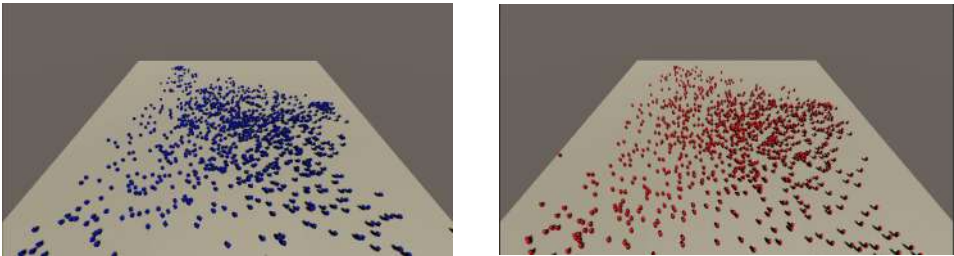
Cas n°1 : Gérer un grand nombre d'objets dans une scène

J'ai choisi ce premier cas, car il peut rapidement survenir lorsqu'un projet commence à prendre de l'ampleur. Mais on peut le retrouver dans les jeux où un très grand nombre d'objets identiques ou similaires est modélisé. Par exemple, dans un jeu de survie où le joueur doit combattre des hordes de zombies, des jeux de stratégies avec des armées immenses ou encore des jeux contenant de la simulation de foule.

Chaque objet dans Unity, appelé GameObject, peut se voir rattacher des scripts qui déterminent leur comportement. Il peut y avoir aussi des composants qui gèrent les collisions, la physique, les textures, ou bien d'autres GameObject. Tous ces composants coûtent des ressources, aussi bien en termes de mémoire que d'utilisation CPU et GPU. Un seul GameObject prend généralement peu de ressources, mais quand est-il lorsque nous avons 10, 100 ou 1000 instances de ce même GameObject ?

Prenons par exemple un "agent", c'est un objet en forme de capsule qui peut se déplacer d'un point à un autre dans un environnement en 3D. L'agent est alors composé d'un collider pour gérer les collisions avec le reste de l'environnement, d'un script qui gère son comportement et un autre script qui gère la navigation de l'agent dans l'environnement. Le comportement donné à l'agent est le suivant : il y a 4 "points d'intérêts" répartis sur la carte et chaque agent a comme cible l'un de ces points. L'agent va se diriger automatiquement vers cette cible et toutes les 3 secondes, sa cible change aléatoirement parmi les points de la carte.

En testant avec 10, 100 et 500 agents, on reste au dessus des 60 images par seconde de manière stable. Cependant, lorsqu'on dépasse les 1000 agents, les IPS chute, mais le jeu reste malgré tout assez stable (autour de 20 IPS). En regardant les stats de la scène, on remarque que le nombre de triangles est grandement influencé par le nombre d'agent. En effet chaque agent est composé d'environ 3300 triangles, ce qui nous fait un total de 3,4 millions de triangles et 8 millions de sommets. On peut facilement avancer qu'il est possible de réduire le nombre de triangle en modifiant la forme de l'agent. Voici un tableau récapitulatif en fonction de la forme et du nombre d'agents.



Mesh	Nombre	IPS	Triangles	Batches
Capsule	1000	20	5.3 M	3200
Cube	1000	25-30	78 K	3000
Capsule	3000	~10	8 M	9600
Cube	3000	~10	100K	8000

On peut facilement remarquer que la fluidité de la scène n'est pas beaucoup impactée par le nombre de triangles et de sommets, ce qui n'est pas le cas de la mémoire avec le nombre de batches. Cependant, en enlevant les colliders sur les agents, j'ai remarqué que l'on peut réduire de plus de moitié le temps de traitement CPU et que l'on augmente les IPS de 10 pour toutes les configurations testées précédemment.

La physique et les interactions entre les objets d'une scène vont grandement impacter la fluidité d'une scène. Tandis que le nombre de triangles et de sommets va impacter la mémoire allouer pour cette même scène. C'est pourquoi, pour réduire le coût en ressources, il peut être judicieux de choisir des modèles comportant moins de triangles. Mais aussi, d'avoir des scripts plus intelligents pour faire éviter autant que possible les collisions et les interactions physiques. Pour cela, il existe des scripts pour simuler de l'intelligence de foule et optimiser le pathfinding.

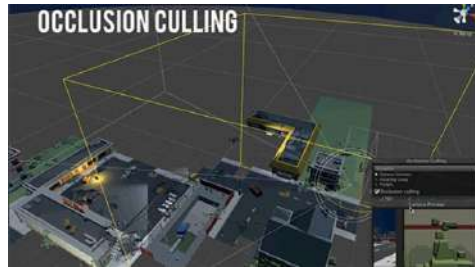
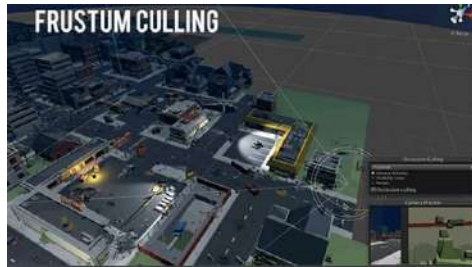
Enfin, une autre manière pour améliorer le temps CPU, est d'optimiser son code. Il est judicieux d'avoir un code optimisé quel que soit le nombre d'objet présent sur la scène. Pour ce faire, il existe des bonnes pratiques qui peuvent être mises en place. L'une d'entre elle étant le fait d'éviter d'utiliser au maximum les boucles Update() et FixedUpdate() qui sont appelés à chaque image du jeu. A la place, il vaut mieux utiliser des appels de fonctions pour déclencher des événements. De manière général, il faut essayer de maintenir un code simple, bien structuré et bien commenté. C'est quelque chose d'important, surtout quand un projet commence à devenir volumineux en terme de code.

Pour résumer ce premier cas, il est possible de gérer un très grand nombre de GameObjects sur Unity. Leur nombre, mais surtout leurs composants vont influencer les performances en jeu.

Cas n°2 : Gérer une scène vaste et complexe

Les jeux contenant de grands environnements avec beaucoup d'objets différents peuvent très rapidement souffrir de pertes de performance. Comme vu précédemment, le nombre de triangle et de sommet jouent un rôle important. Pour y remédier, il existe de nombreuses techniques et de bonne pratiques d'optimisation.

Une première technique consiste à utiliser une méthode de sélection des objets, appelée Frustum Culling. Elle permet d'uniquelement gérer l'affichage des objets se trouvant dans l'angle de vue de la caméra du joueur. Cela permet de grandement réduire le nombre de triangles et de sommets à modéliser dans la scène. Cependant, cette technique affiche quand même des objets qui sont dissimulés par d'autres objets dans la scène, ce qui consomme de la mémoire. C'est pourquoi il existe une technique similaire au Frustum Culling, mais qui prend en compte le fait que des objets peuvent être cachés par d'autres, c'est l'Occlusion Culling. Ainsi, cette seconde technique de sélection permet de réduire encore davantage le nombre de triangles à modéliser.



Cependant, même si ces deux techniques sont des solutions efficaces, elles ont quand même quelques limitations. En effet, tous les GameObjects qui ne sont pas affichés sont malgré tout stockés en mémoire, il faut donc s'assurer d'avoir suffisamment de mémoire disponible. De plus, le chargement et le déchargement du rendu des objets consomment du CPU, surtout l'Occlusion Culling, car il y a plus d'objets qui sont concernés.

En bref, les méthodes de culling sont très utiles, mais elles sont plus efficaces dans certaines scènes que d'autres. Par exemple, l'occlusion culling est très efficace à l'intérieur des bâtiments car beaucoup d'objets peuvent être cachés par des murs ou des portes.

Il est possible d'optimiser le rendu d'une scène en prenant soin d'analyser chacun des objets et comment ils influent sur les performances. Pour cela, on peut simplement retirer tous les objets et les rajouter un à un tout en regardant le profiler d'Unity. Cette méthode est très utile, par exemple, lorsqu'on utilise des assets provenant de l'Assets Store de Unity. Les assets sont des packages créés par d'autres personnes, qui peuvent être téléchargés et importés dans un projet Unity. Cependant, il arrive que ces assets ne soient pas bien optimisés et qu'ils consomment plus de ressources que nécessaire.

Pour illustrer la démarche à suivre, j'ai pris un asset contenant une scène qui modélise une grande forêt sur un terrain avec divers dénivelés, de nombreux rochers, de l'herbe, des plantes, des maisons et d'autres objets de décoration. Sans rien modifier, la scène a les statistiques suivantes : autour de 120 IPS, 1.2 million de triangle, 1.8 million de sommets et 2730 batches.



Pour voir si il est possible de réduire le nombre de triangles, de sommets et de batches, on peut commencer par identifier quels objets pèsent le plus lourd. On peut répartir les objets de la scène en deux catégories, la première étant le terrain qui va contenir les arbres, les plantes et l'herbe. La deuxième étant tous les autres objets de décors, c'est-à-dire les rochers, les maisons et tous les autres petits objets.

	Triangles	Sommets	Batches
Total	1.2 M	1.8 M	2730
Décorations	~650 k	~500 k	~1500
Terrain	~750 k	~1.3 M	~1200

Tout d'abord, on remarque que c'est le terrain qui prend le plus de ressources. Une première étape serait d'éditer les paramètres du terrain pour réduire la qualité des graphismes. En réduisant la distance d'affichage à laquelle les arbres ont leur meilleure qualité graphique, on gagne déjà en performance sans pour autant réduire la qualité globale. En effet les arbres plus éloignés du joueur seront de moins beau, mais ils sont suffisamment hors de portée pour que ce ne soit pas visible par le joueur. On peut faire la même chose pour l'herbe en réduisant leur distance d'affichage. Il est aussi possible d'augmenter légèrement les détails des touffes d'herbes pour réduire le nombre de batches au prix de quelques triangles en plus.

Ensuite, on peut réduire la résolution du terrain, ce qui correspond au nombre de polygones qui le compose. Comme il est en grande partie dissimulé par l'herbe et le décor, cela ne devrait pas être visible. Pour les mêmes raisons, on peut réduire la distance d'affichage des détails du terrain.

Pour optimiser les objets de décorations, une première chose à faire est de les marquer en tant qu'objet "static", étant donné qu'ils n'ont pas à bouger. Pour les objets complexes qui sont composés de plusieurs sous-objets, comme les bâtiments, cela permet de grandement réduire le nombre de batches. Tandis que pour les objets de plus petites tailles comme les lanternes, il arrive souvent que la résolution de leur texture soit plus grande que nécessaire. Il suffit donc de la réduire, pour gagner un peu en mémoire, sans changer la qualité visuelle globale.

Les rochers sont des objets qui contiennent beaucoup de polygones. Une solution pour réduire leur nombre, est de les remplacer par un modèle qui en comporte moins lorsque le joueur est éloigné du rocher.

Pour finir, on peut appliquer des techniques un peu plus générales, tel que l'occlusion culling, que j'ai présenté précédemment. Il est aussi possible de réduire la qualité des ombres, la résolution des réflexions et d'opter pour des spots de lumière par vecteur plutôt que par pixel. En bref, toutes ces techniques visent à améliorer les performances et peuvent être appliquées dès le début d'un projet. Pour vous montrer à quel point ces techniques sont efficaces, voici un tableau comparatif des performances avant et après l'optimisation :

	Triangles	Sommets	Batches
Avant optimisation	1.2 M	1.8 M	2730
Après optimisation	218 k	282.4 k	209

Comme on peut le voir, les performances ont été grandement améliorées, sans pour autant trop réduire la qualité globale de la scène. En effet, en comparant deux captures d'écran, avant et après l'optimisation. Mis à part les ombres, il y a peu de différences.



Avant optimisation



Après optimisation

Conclusion

Pour conclure sur cette deuxième partie du dossier créatif, j'ai beaucoup apprécié les recherches et les expérimentations que j'ai pu faire sur l'optimisation des performances dans les jeux vidéo. Comme on peut s'en douter, ce sujet est immensément vaste et peut énormément varier d'un jeu à l'autre, ou même d'un moteur de jeu ou d'une plateforme à l'autre.

Dans la partie précédente, on peut constater que les améliorations de performances que j'ai pu effectuer sur la scène sont assez simples à mettre en place. Cependant, appliquer de telles techniques peut impliquer malgré tout une baisse de qualité graphique, comme pour les ombres dans notre cas. Cela peut être un véritable challenge pour les jeux qui veulent miser sur une excellente qualité graphique, en utilisant par exemple des textures détaillées. Tandis que d'autres jeux vont chercher à utiliser des techniques plus coûteuses pour augmenter la qualité de la lumière, des ombres et des réflexions. L'utilisation de telles techniques va nécessiter l'utilisation de matériel plus performant en termes de composant par le joueur pour qu'il puisse jouer au jeu. Pour résumer, il faut trouver un équilibre entre qualité et performance et souvent, il faut faire des compromis.

Enfin, même si cette problématique n'est pas innovante à proprement parler, elle est présente depuis toujours dans le monde du jeu vidéo. Aussi, elle fait toujours l'objet de nombreuses recherches, donnant ainsi naissance à de nouvelles techniques d'optimisation, qui sont à leur tour utilisées dans la conception de jeux vidéo.